

Aptos

이준서



목차

- 블록체인 업계에서 Aptos의 의의
 - 블록체인의 역사
- Move Language
- Aptos 병렬 실행

Aptos의 의의



블록체인의 역사

- 1세대 블록체인 : Bitcoin
 - 최초의 디지털 화폐로 거래 내역을 중앙 서버가 아닌 분산 원장에 기록
 - 탈중앙화, 데이터 무결성, 보안
 - 확장성 문제 및 에너지 소모 크고 화폐 거래에 초점 맞춰져 스마트 계약 부재 및 거래 지연
- 2세대 블록체인 : Ethereum, Neo, ...
 - Smart Contract를 통해 블록체인의 기능 확장
 - 특정 조건 충족되면 자동으로 실행되는 중개자 없이 신뢰 가능한 거래 (Solidity)
 - 블록체인 트릴레마 문제
- 3세대 블록체인 : Solana, Polkadot, Cosmos, Aptos, ...
 - 이전 세대의 블록체인이 가진 문제를 해결하기 위해 설계된 블록체인
 - 병렬처리와 샤딩, 레이어 설계, Cross-Chain Protocol 등의 기술 도입

Why Blockchain

- Web2 Internet에서는 중앙 집중식 회사가 사용자 데이터를 직접 제어
 - Google, Amazon, Apple, Meta, ...
 - 클라우드 인프라를 통해 수십억 명의 사용자로 확장할 수 있는 서비스 구축 가능
 - Web2에서는 사용자들이 중앙화된 기업에 대한 신뢰가 필요해서 우려가 높아짐
- 블록체인을 사용한 새로운 인터넷 시대 Web3 시작
 - 사용자가 중앙화된 기업에 대한 신뢰 요구 받지 않고 안전하고 안정적으로 상호작용 가능
 - 데이터 소유권 회복
- 하지만 아직 Web3 광범위 채택 이루어지지 않음
 - 신뢰성 낮고 높은 거래 수수료, 처리량 한계, 보안 문제, 실시간 대응 지원X

블록체인 트릴레마

- 세 가지 목표 가운데 두 가지 목표는 동시에 해결할 수 있지만 세가지 목표를 한꺼번에 해결할 수 없는 문제
 - 확장성(Scalability)
 - 블록체인이 얼마나 많은 트랜잭션을, 얼마나 빠르게 처리할 수 있는지
 - 탈중앙화(Decentralization)
 - 중앙화된 집단이 통제하는 것이 아닌 서로 다른 개별 참여자들이 합의를 통해 네트워크 운영 및 관리할 수 있는지
 - 보안(Security)
 - 네트워크에 대한 공격으로부터 블록체인에 기록된 정보 및 자산 안전하게 지킬 수 있는지

Aptos의 트릴레마 문제 해결 시도

➤ 확장성

▪ 병렬 실행

- 트랜잭션 간 데이터 충돌 관리 메커니즘

▪ 블록 파이프라이닝

- 블록 생성, 전파, 합의 과정을 병렬로 실행하여 블록 생성 속도 최적화
- 트랜잭션 Finality 시간 단축

▪ Sharding

- 수평적 처리량 확장성

➤ 탈중앙화

▪ PoS

➤ 보안

▪ Move Language 사용

- Resource-Oriented Programming

▪ AptosBFT consensus

- Hotstuff 기반의 BFT 합의 알고리즘

Aptos 비전

- The safest and most scalable Layer 1 blockchain
- Web3, DApp을 메인 스트림으로 가져오는 것 목표
- 유연하고 모듈화된 블록체인 아키텍처 제공
- 새로운 사용 사례와 기술 진보에 따라 사용자 방해하지 않는 자연스러운 업그레이드

Aptos 설계 원칙

- 새로운 스마트 컨트랙트 언어 Move
- batched, pipelined, parallelized 트랜잭션 처리
- Block-STM 병렬 트랜잭션 처리로 Atomicity 효율적 지원
- stake-weight validator set rotation and reputation tracking
- sharding

Move PL



Aptos Blockchain 구현 언어

➤ Move 구현 영역

- Smart Contract
- 트랜잭션 처리 및 상태 변경
 - 트랜잭션에 따른 리소스의 이동, 검증, 결과 저장
- 스마트 계약 검증 및 실행

➤ Rust 구현 영역

- Network core
 - 노드, 네트워크 통신, 데이터 저장
- 합의 알고리즘

Move Language

- Executable bytecode language used to implement custom transactions and smart contracts
 - Bytecode : 플랫폼 독립적으로 가상 머신에서 실행되도록 설계된 코드
 - Java, Move, Solidity
 - 동일한 Bytecode를 다양한 플랫폼에서 실행할 수 있어서 블록체인과 같은 분산환경에서 유용
- Solidity + Rust
 - Solidity의 스마트 계약 안전성은 프로그래머의 코드 작성에 따라 다름
 - Move는 Rust의 소유권 기반 Resource 모델 방법으로 안전성 강제로 보장
 - on-chain bytecode verifier인 Move Prover을 통해 프로그램 검증

Solidity 예제

- Account-based
 - 계정 주소와 토큰 잔액을 mapping
- Safety
 - require을 사용해서 조건을 검사
 - 실패 시 transaction rollback
- State Transition
 - 잔액을 직접 수정하는 형식의 상태 변경
- 상태 변경에 따른 가스 비용 발생함

```
2  pragma solidity ^0.8.0;
3
4  contract Token {
5      mapping(address => uint256) public balances;
6
7      constructor(uint256 initialSupply) {
8          balances[msg.sender] = initialSupply;
9      }
10
11     function transfer(address recipient, uint256 amount) public {
12         require(balances[msg.sender] >= amount, "Insufficient balance");
13         balances[msg.sender] -= amount;
14         balances[recipient] += amount;
15     }
16 }
17
```

Move 예제

- Token 모듈
 - 블록체인의 주소 0x1에 정의
- move_to()
 - account의 storage에 리소스 이동
 - 복사되지 않고 소유권이 계정으로 이동
- borrow_global_mut()
 - sender의 Balance를 가변 참조 가져옴
 - RUST와 동일한 방법
 - ✓ 동일한 시점에 하나의 가변 참조만 가능 보장
 - 불러온 가변참조에 대해서만 수정 가능

```

1 module 0x1::Token {
2     resource struct Balance {
3         value: u64,
4     }
5
6     public fun create_balance(account: &signer, initial_supply: u64) {
7         move_to(account, Balance { value: initial_supply });
8     }
9
10    public fun transfer(sender: &signer, recipient: address, amount: u64) {
11        let sender_balance = borrow_global_mut<Balance>(Signer::address_of(sender));
12        assert!(sender_balance.value >= amount, 1);
13
14        sender_balance.value -= amount;
15
16        if (exists<Balance>(recipient)) {
17            let recipient_balance = borrow_global_mut<Balance>(recipient);
18            recipient_balance.value += amount;
19        } else {
20            move_to(&recipient, Balance { value: amount });
21        }
22    }
23 }

```

Move 언어의 안전성

➤ Resource 중심 설계

- 복제, implicit 삭제 불가능한 특별한 데이터 타입 Resource를 사용하여 자산 모델링
- 소유권 이동
 - Storage 위치 이동만 가능하게 함
- Borrowing 시스템
 - 가변 참조, 불변 참조를 명확히 구분
 - ✓ Data Race 방지하여 integrity 보장
 - 명시적으로 가져와야만 상태를 수정할 수 있음

➤ 정적 검증

- Resource 중심 설계로 컴파일 타임에 자산의 Integrity 검증할 수 있음
 - Type system을 통해 컴파일 타임에 방지 가능
 - Resource 복제되지 않고 다시 접근되지 않도록 보장

Aptos 병렬 실행



Blockchain의 성능 병목 문제

➤ Sequential Execution

- **Conflict 방지하고 Consistency 유지하지만 TPS 매우 낮음**
 - 동기화 비용이 높음
- **EVM, 기존 PoW, PoS 방식**
 - 트랜잭션 독립적으로 처리하고 모든 노드에서 일관되게 유지
 - 비트코인: 평균 7TPS, 블록 생성 시간 10분
 - 이더리움: 평균 20~30TPS
- **병렬 실행 기술 적용하면 TPS 극대화**
 - 앵토스: 테스트환경 기준 16만TPS
 - ✓ 실제 메인넷에서는 1000~4000TPS 수준

병렬 처리를 위한 과제

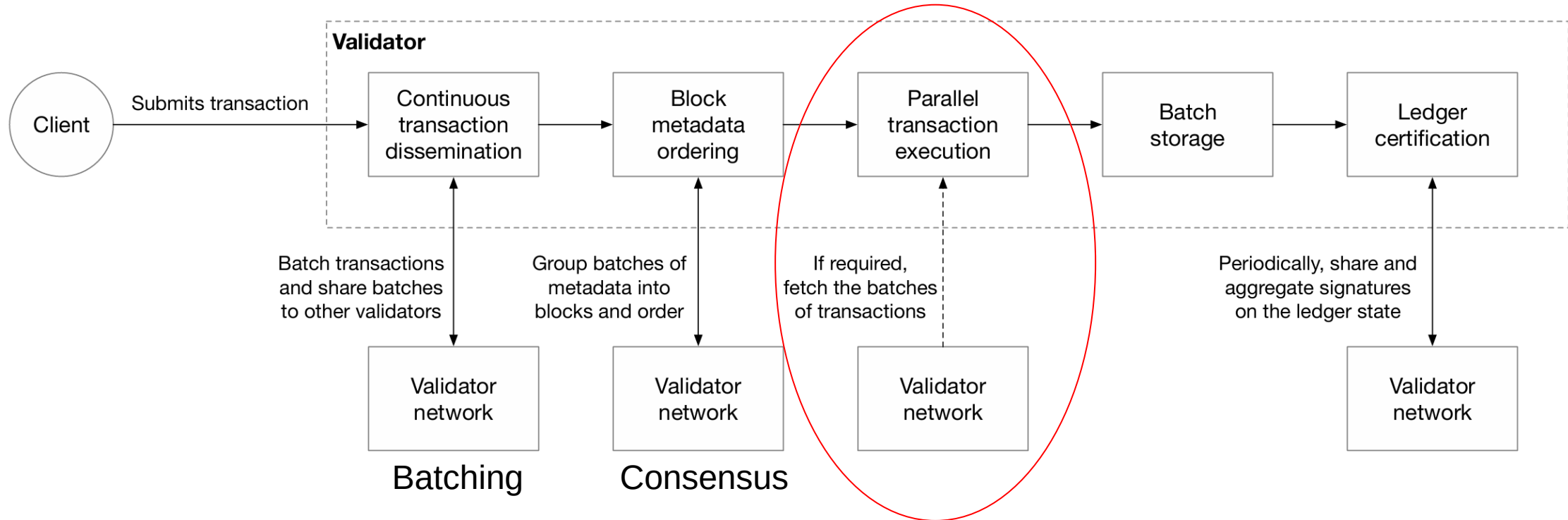
- 트랜잭션 간 데이터 의존성
 - 트랜잭션 간 데이터 Read/Write 작업 중복될 경우 Conflict 발생
- Deterministic Outcomes
 - 블록체인의 주요 특징: 동일한 트랜잭션 실행했을 때 모든 노드에서 동일한 결과 보장
 - 실행 순서 변경되어도 항상 같은 결과를 보장

기존 병렬 실행 방법

- 트랜잭션 간 의존성을 사전 계산하여 DAG 형태로 표현
 - 의존성 인식하여 충돌 방지
- Deterministic DB인 BOHM
 - write-sets 실행 전에 이미 알려져 있다고 가정할 때 특정 직렬화 순서 강제
 - multi-version data structure 사용해서 read-dependency 해결
 - read를 해당 데이터의 가장 최신의 write와 연결

Aptos 트랜잭션 실행 Life Cycle

- 전파된 트랜잭션 검증자에게 전파하고 Batch
- Batch들을 Block Metadata로 변환하여 그룹화, 정렬
- 병렬 트랜잭션 실행
- Batch 저장하고 Ledger 검증



Parallel execution engine

➤ Block-STM 병렬 실행 엔진

- STM 원칙 기반으로 설계된 in-memory Smart Contract 병렬 실행 엔진
- 실행 도중 발생할 수 있는 트랜잭션 간 의존성 동적으로 감지
- **Optimistic Concurrency Control**
 - 트랜잭션 실행된 이후 메모리 접근 검증하고 검증 중에 충돌 발생 시 트랜잭션 중단하고 재실행
- **multi-version data structure** 사용해서 미리 정의된 직렬화 강제
 - 최종 결과는 순차적으로 실행한 것과 동일
- **write-sets** 사전 지식, 사전 계산, 추가적인 통신 없이 트랜잭션 실행 자동으로 가속화

Software Transactional Memory(STM)

- 트랜잭션 기반 메모리 접근으로 동기화
 - 공유 메모리에 대한 동시 접근 안전하게 관리
 - Read-set, Write-set 집합 사용하여 트랜잭션 간 충돌 여부 판단
 - OCC
- 작동 과정
 - 1. 트랜잭션 시작
 - 2. 트랜잭션 실행
 - Read-set: 메모리에서 값 읽고 추가
 - Write-set: 메모리 값 변경하지 않고 변경 내용 버퍼링
 - 3. Validation
 - 4. Commit or Rollback

Optimistic Concurrency Control(OCC)

- 트랜잭션 수행 동안은 어떠한 검사도 하지 않고 종료 시에 일괄적으로 검사
- 종료 시에 트랜잭션 직렬화 검증되면 DB에 반영
 - 여러 트랜잭션이 Conflict 없다고 가정하고 검증 단계에서 Conflict 있으면 abort, re-execute

트랜잭션 처리 예시

➤ Sequential Execution

- TX1-TX2-TX3-TX4-TX5-TX6-TX7-TX8-TX9-TX10

➤ 의존성 있는 트랜잭션 병렬 처리

▪ 의존성

- TX1-TX2-TX3-TX4
- TX3-TX6
- TX3-TX9

- TX1 ->TX2 재시도-> TX3재시도 -> TX4 재시도
- TX2 abort ->TX5 ->TX8 ->TX6 재시도
- TX3 abort ->TX6 abort -> TX6 재시도 abort ->TX9 재시도
- TX4 abort ->TX7 -> TX9 재시도 ->TX10

Multi-version Concurrency Control

- 동일한 위치에 대한 모든 쓰기는 트랜잭션 ID, 다시 실행된 횟수 포함하는 version으로 저장
- 데이터의 여러 version을 유지하여 동시성 지원
 - 데이터의 변경 사항을 새로운 version으로 저장
 - 각 트랜잭션은 데이터의 snapshot에 대한 읽기 및 쓰기 작업 수행
 - 트랜잭션 간 읽기/쓰기 충돌 최소화
- BOHM에서는 Read-set, Write-set을 미리 계산하여 의존성 사전에 결정
 - preset Serialization하고 순서대로 실행됨
- Block-STM에서는 Write-set 정보 사전에 알 필요 없음
 - 실행 중 충돌 탐지, 해결하는 OCC 사용하여 동적 직렬화

Block-STM이 write sets 사전 정보 없어도 되는 이유

- OCC로 실행 중에 동적으로 write set 생성하고 검증 중에 충돌 여부 판단
- Multi version 데이터 구조로 동일한 데이터에 대해 여러 버전 유지하여 충돌 회피 가능

Block-STM 성능 기여

➤ Block 단위 업데이트

▪ Lazy Commit

- 트랜잭션 결과를 독립적으로 Commit하지 않고 Block 단위로 Commit
- 동기화 비용 줄고 병렬 실행 성능 극대화

➤ Double-Collect

▪ Execution, Validation 단계 분리

▪ Collaborative Scheduling

- Execution, Validation을 스레드간에 동적으로 배분
- 의존성 detect, estimate하여 pending 시키고 다음 수행 트랜잭션 선택하는 스케줄링

➤ 구현 결과

- 32쓰레드 기준 17~20배 빠른 성능
- BOHM 모델에 DAG 완벽히 제공한 상태에서 실험 결과와 비슷한 성능

다음 발표

➤ Sui

- Move language 사용방법에서의 차이점
 - 계정 기반 모델과 객체 기반 모델
- 확장성 문제 해결방법의 차이점

Aptos 한계점

- 탈중앙성 철학 해침
 - 고성능의 검증 노드 스펙 필요
 - CPU: 8코어 이상
 - RAM: 32GB 이상
 - Storage: 1TB이상 SSD
 - 네트워크: 최소 1Gbps 이상

Parallel data model

➤ 트랜잭션 Conflict 없으면 병렬 실행 가능

▪ Move data model은 global addressing 지원

- 트랜잭션 간 독립성 보장
- 데이터 충돌 감지 및 회피 가능

▪ 서로 다른 트랜잭션이 각각 다른 계정에 접근하거나 다른 데이터 수정하면 병렬로 실행 가능

➤ Delta-Write

▪ 트랜잭션이 실행되면서 상태 즉시 변경하지 않고 변경 사항을 delta 형식으로 저장

- 중간 상태 저장하고 실행 모두 완료된 후 델타 값 모아서 최종 상태 반영
- conflicting 값에 대해서는 동일한 순서로 적용

S

▶ 예제

S

▶ 예제

B. Sample Pages illustrating figure related errors

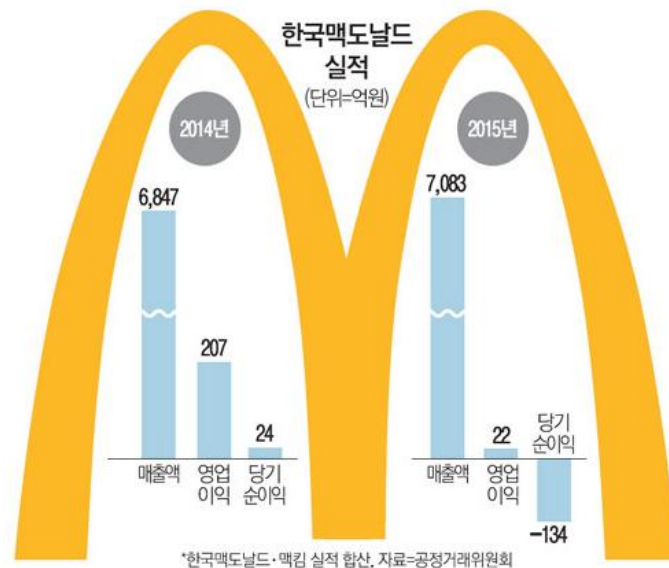


Sample Page1

- 글과 연관 있는 그림만 삽입하며 가급적 캡션 추가
- Put the figures related to the writing only, and add captions whenever possible

➤ Example

- McDonald's has a lots of branches in the world



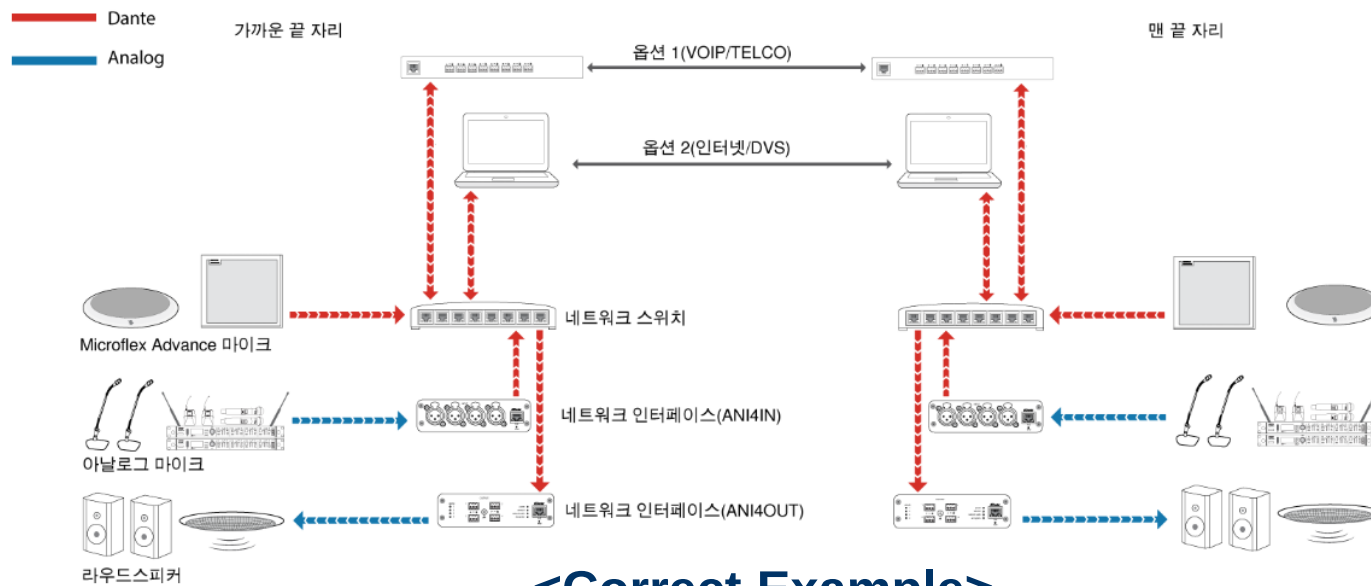
<Wrong Figure>



<Correct Figure>

Sample Page2

- 선의 목적을 명확하게 드러내라
- You should clarify the purpose of the lines.
- Example
 - The example shows the overall signal path of the networked conferencing system



<Correct Example>

Precautions



Precautions

- 발표자료는 최소 1주 전에는 완성하여 최소 2번 이상의 수정작업을 거쳐야함
 - 한 페이지에 한 주제만 담음
 - 한 페이지에서 중요한 내용은 최대한 제목과 중앙에 집중시킴
 - PPT의 모든 페이지들은 동일한 양식을 사용
 - 애니메이션을 사용할 때 입과 행동을 맞춰야 함
 - 한 페이지는 많은 내용을 담으려고 하지 않음
-
- Presentations must be completed at least one week and must have a minimum of two modifications
 - Contain only one subject per page
 - Focus on important content on one page in the title and center as much as possible
 - All pages in PPT should use the same form
 - You have to match your mouth and action when using an animation
 - Do not contain too much information in one page